Activity 2:

User Story 1: Reject Invalid Task Input

As a student, I want the system to reject invalid task input so that only well-created tasks are saved.

Test Case 1.1: Submit task with different titles

Precondition: User is logged in and on the Planner page.

Steps:

- Log in to ScheduleLynx and navigate to the Planner page.
- Set the *Title* field as:
  a) “” // empty case
  b) “ ” // whitespace case
  c) A // valid min case
  d) ABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWX // invalid max case
- Set *Type* to “Task”, enter a valid future date, and set *Estimated Hours* to 2.
- Click “Save Task”.

Expected Result:

a) The system should display a validation error and not save the task.
b) The system should display a validation error and not save the task.
c) The system should not display any validation error and save the task.
d) The system should display a validation error and not save the task.

Actual Result:

a) The form prevented a submission with an empty title. No task is saved due to the message “Please fill in the field”.
b) The browser prevented a submission with an empty title. No task is saved due to the message “Please fill in Title, Type, and Date.”.
c) The task was saved
d) The task was saved

Status:

a) Passed
b) Passed
c) Passed
d) Failed

Test Case 1.2: Submit task with estimated hours set at different values

Precondition: User is logged in and on the Planner page.

Steps:

- Navigate to the Planner page.
- Enter a valid title (i.e. “Hours Test Task”).
- Set *Type* to “Task” and enter a valid future date.

a) Set *Estimated Hours* to -5. // invalid negative hours case
b) Set *Estimated Hours* to 0. // invalid 0 hours case
c) Set *Estimated Hours* to 1. // valid min hours case
d) Set *Estimated Hours* to 5. // valid hours case
e) Set *Estimated Hours* to 24. // valid max hours case
f) Set *Estimated Hours* to 28. // invalid +24 hours case

- Click "Save Task".

Expected Result:

a) The system should reject the submission and display an error message.
b) The system should reject the submission and display an error message.
c) The system should accept the submission.
d) The system should accept the submission.
e) The system should accept the submission.
f) The system should reject the submission and display an error message.

Actual Result:

a) A message. "Estimated hours must be at least 1 for a Task." pops and the task is not saved.
b) A message. "Estimated hours must be at least 1 for a Task." pops and the task is not saved.
c) No message pops and the task is saved.
d) No message pops and the task is saved.
e) No message pops and the task is saved.
f) No message pops and the task is saved.

Status:

a) Passed
b) Passed
c) Passed
d) Passed
e) Passed
f) Failed

Test Case 1.3: Submit task with different date

Precondition: User is logged in and on the Planner page.

Steps:

- Navigate to the Planner page.
- Enter a valid title (i.e "Date Test Task").
- Set *Type* to "Task".
- Set the Date field as:
  a) null // empty case
  b) past date // 2026-03-27
  c) current date // 2026-03-29
  d) future date // 2026-03-31
  e) far future date // i.e. 2900-01-01
- Enter *Estimated Hours* as 1.
- Click "Save Task".

Expected Result:

a) The system should reject the task and show a validation error.
b) The system should reject the task and show a validation error.
c) The system adds the task.
d) The system adds the task.
e) The system should reject the task and show a validation error.

Actual Result:

a) The form prevented a submission with no date. No task is saved due to the message, "Please fill in the field".
b) The task is saved and added.
c) The task is saved and added.
d) The task is saved and added.
e) The task is saved and added.

Status:

a) Passed
b) Failed
c) Passed
d) Passed
e) Failed

Test Case 1.4: Submit task with different min and max block hours

Precondition: User is logged in and on the Planner page.

Steps:

- Navigate to the Planner page.
- Enter a valid title (i.e. "Block Hours Test Task").
- Set *Type* to "Task", enter a valid future date, and set *Estimated Hours* to 1.
- Set:
    a) *Min Block Hours* to null and *Max Block Hours* to null. // null case
    b) *Min Block Hours* to 0 and *Max Block Hours* to 0. // zero edge case
    c) *Min Block Hours* to 1 and *Max Block Hours* to 0. // missing max case
    d) *Min Block Hours* to 0 and *Max Block Hours* to 1. // missing min case
    e) *Min Block Hours* to 2 and *Max Block Hours* to 2. // valid equal hours case
    f) *Min Block Hours* to 2 and *Max Block Hours* to 5. // valid min < max case
    g) *Min Block Hours* to 5 and *Max Block Hours* to 2. // invalid min > max case
- Click "Save Task".

Expected Result:

a) The system should reject the submission and an error message should occur.
b) The system should reject the submission and an error message should occur.
c) The system should reject the submission and an error message should occur.
d) The system should reject the submission and an error message should occur.
e) The system accepts and adds the submission.
f) The system accepts and adds the submission.
g) The system should reject the submission and an error message should occur.

Actual Result:

a) No error message and task is accepted // automatically assigns Min: 1 hour & Max: 3 hours
b) The message "Value must be greater than or equal to 1." is shown and the task is not saved.
c) The message "Value must be greater than or equal to 1." is shown and the task is not saved.
d) The message "Value must be greater than or equal to 1." is shown and the task is not saved.
e) The task is added.
f) The task is added.
g) The message "Task min block hours cannot be greater than max block hours." is shown and the task is not saved.

Status:

a) Failed
b) Passed
c) Passed
d) Passed
e) Passed
f) Passed
g) Passed

Test Case 1.5: Submit event with different times

Precondition: User is logged in and on the Planner page.

Steps:

- Navigate to the Planner page.
- Enter a valid title (i.e. "Event Times Task").
- Set *Type* to "Event: and enter a valid future date.
- Set time as:
    a) Start Time: null and End Time: null. // empty case
    b) Start Time: 3:00 AM and End Time: null. // invalid case
    c) Start Time: null and End Time: 3:00 AM. // invalid case
    d) Start Time: 3:00 AM and End Time: 2:00 AM. // invalid case
    e) Start Time: 3:00 AM and End Time: 3:00 AM. // invalid case
    f) Start Time: 2:00 AM and End Time: 3:00 AM. // valid case
- Click "Save Task".

Expected Result:

a) The system should not add the task and show a message.
b) The system should not add the task and show a message.
c) The system should not add the task and show a message.
d) The system should not add the task and show a message.
e) The system should not add the task and show a message.
f) The system should add the task.

Actual Result:

a) The message, "Please enter start and end time for an Event." is shown and no task is added.
b) The message, "Please enter start and end time for an Event." is shown and no task is added.
c) The message, "Please enter start and end time for an Event." is shown and no task is added.
d) The message, "End time must be after start time." is shown and no task is added.

e) The message, “End time must be after start time.” is shown and no task is added.
f) The task is added.

Status:

a) **Passed**
b) **Passed**
c) **Passed**
d) **Passed**
e) **Passed**
f) **Passed**

User Story 2: Add a Fixed (Recurring) Event

As a student, I want to be able to add a fixed recurring event so that my regular commitments are reflected in my schedule.

Test Case 2.1: Add a daily recurring event

Precondition: User is logged in and on the Planner page.

Steps:

- Navigate to the Planner page.
- Enter a title (i.e. “Morning Run”).
- Set *Type* to “Event”, enter a start date (i.e. today's date), Start Time as 07:00 AM and End Time as 08:00 AM.
- Check “Recurring”, select *Recurrence Type* as “Daily”.
- Set the *Recurrence End Date* one week from now.
- Click “Save Task”.

Expected Result: The event should be saved and appear on every day from the start date through the recurrence end date on the calendar.

Actual Result: The event saves successfully and is shown everyday from the start date to the end date.

Status: **Passed**

Test Case 2.2: Add a weekly recurring event on specific days

Precondition: User is logged in and on the Planner page.

Steps:

- Navigate to the Planner page.
- Enter a title (i.e. “EECS 2311 Lecture”).
- Set *Type* to “Event”, enter a start date that falls on a Monday, Start Time as 09:00 AM and End Time as 11:00 AM.
- Check “Recurring”, select *Recurrence Type* as “Weekly”.
- Check “Mon” and “Wed” from the days-of-week checkboxes.
- *Set a Recurrence End Date* three weeks from the start date and on a Tuesday.
- Click “Save Task”.

Expected Result: The event should appear on every Monday and Wednesday from the start date through the recurrence end date on the calendar.

Actual Result: Event saves and appears on the Mondays and Wednesdays of the week based on the recurrence logic. For the last week, only the Monday event is shown.

Status: **Passed**

Test Case 2.3: Add a biweekly recurring event
Precondition: User is logged in and on the Planner page.

Steps:
- Navigate to the Planner page.
- Enter a title (e.g., "Biweekly Chess Meeting").
- Set *Type* to "Event", enter a valid start date, Start Time as 01:00 PM and End Time as 02:00 PM.
- Check "Recurring", select *Recurrence Type* "Biweekly".
- Select "Fri" as the recurrence day.
- Set a *Recurrence End Date* four weeks out.
- Click "Save Task".

Expected Result: The event should appear on every other Friday starting from the start date's week and through the recurrence end date.

Actual Result: Event saves and appears after every 2nd Fridays until the last week.

Status: **Passed**

User Story 3: Edit an Existing Event
As a student, I want to edit an existing event so that I can update its details when things change.

Test Case 3.1: Edit an event/task's title and time
Precondition: User is logged in and at least one saved event exists.
Steps:

- Navigate to the Planner page.
- Click on an existing event or task card to open its details under *Your Tasks*.
- Click "Edit".
- Change the title to a new value and update the start/end times.
- Click "Update Item".
- Find the updated event and open its details.

Expected Result: The updated title and times should be reflected correctly in the task list and calendar.

Actual Result: The updated event or task was reflected correctly.

Status: **Passed**

Test Case 3.2: Edit a non-recurring event to become a recurring event
Precondition: User is logged in and at least one non-recurring event exists.

Steps:
- Navigate to the Planner page.
- Click on a non-recurring event to open its details under *Your Tasks*.
- Click “Edit”.
- Check the “Recurring” checkbox.
- Select *Recurrence Type* as “Weekly” and check “Tue”.
- Set a *Recurrence End Date* two weeks out.
- Click “Update Item”.

Expected Result: The event should now appear as a recurring weekly event on Tuesdays.

Actual Result: The edit saves and the recurrence appears on Tuesdays.

Status: **Passed**

Test Case 3.3: Edit event with conflicting time
Precondition: User is logged in and two events exist on the same date, where Event A (08:00 to 09:00) and Event B (10:00 to 11:00).

Steps:
- Navigate to the Planner page.
- Click on Event B and click “Edit” under *Your Tasks*.
- Change Event B's start time to 08:30 and end time to 09:30.
- Click “Update Item”.

Expected Result: The system should detect a time conflict with Event A and reject the save and a conflict warning is shown.

Actual Result: The task is not saved and the conflict error is showing the message “TIME CONFLICT: "Event A" (08:00 - 09:00) is already scheduled on Mar 29, 2026. Please choose a different time.”.

Status: **Passed**

Test Case 3.4: Edit event Type from Event to Task
Precondition: User is logged in and an event exists.

Steps:
- Navigate to the Planner page.
- Open an event's box and click “Edit” under *Your Tasks*.
- Change the *Type* from “Event” to “Task”.
- Fill in the *Estimated Hours* (i.e. 2).
- Click “Update Item”.

Expected Result: The original event should be replaced as a task entry.

Actual Result: The event instantly becomes a task following the update. The same is true for Task to Event as well.

Status: **Passed**

Bug Reports written on GitHub:

Key summaries

- Invalid Date Acceptance (#44)
    - Example: A user enters a title like "ABCDEFGHIJKLMNOPQRSTUVWXYZ…" (a very long string)
    - Expected: System to reject it due to exceeding max length.
    - Actual: The task is saved successfully with no error.

- Estimated Hours Upper Bound Not Enforced (#43)
    - Example: A user sets *Estimated Hours* = 28.
    - Expected: System to reject since it exceeds the 24-hour limit.
    - Actual: The task is saved with 28 hours and no validation message appears.

- Title Length Validation Missing (#42)
    - Example 1: A user selects a past date (2026-03-27).
    - Expected: System to reject it.
    - Actual: The task is saved.
    - Example 2: A user selects a far future date (2900-01-01).
    - Expected: System to reject since it is far out-of-range.
    - Actual: The task is saved.

Activity 3:

Code smells:

Smell 1:
Long Method: addTask() in script.js
Category: Bloater

Description: The addTask() function spans well over 100 lines and handles input retrieval, validation for events, tasks, & recurrence, conflict detection, and API calls. It is a "Long Method" smell since it has too many responsibilities and is difficult to maintain, test, or extend.

Current code:

```
async function addTask() {
  // ~15 lines of value extraction
  // ~10 lines of shared validation
  // ~15 lines of task-specific validation
```

```
  // ~20 lines of event-specific validation
  // ~40 lines of branching API calls
  // ~10 lines of post-save refresh
}
```

Proposed Fix: Split addTask() into focused helpers

```
Function getFormValues() { /* extraction */ }
function validateTaskInput(values) { /* task-only validation */ }
function validateEventInput(values) { /* event-only validation */ }
async function saveTaskToBackend(values, originalItem) { /* task API
logic */ }
async function saveEventToBackend(values, originalItem) { /* event API
logic */ }
async function addTask() {
  const values = getFormValues();
  if (values.type === "task") {
    if (!validateTaskInput(values)) return;
    await saveTaskToBackend(values, originalItem);
  } else {
    if (!validateEventInput(values)) return;
    await saveEventToBackend(values, originalItem);
  }
  await refreshAfterSave();
}
```

Smell 2:
Duplicate Code: UI refresh calls repeated in multiple functions
Category: Dispensable / Duplicated Code

Description: The same sequence of UI refresh calls appears copy-pasted in at least three separate async functions (addTask, deleteTaskListener, clearAllItems):

Current code:

```
await loadUserTasks(currentUser.username);
await loadScheduleEntries();
updateTasksDisplay();
renderCurrentScheduleView();
renderTimeline(scheduleEntries);
refreshDashboardIfVisible();
```

The “Extract Method" refactoring could be applied. If the refresh logic ever needs to change (i.e. adding a new render step), it can be updated in every location, ensuring a hazard in maintenance doesn’t take place.

Proposed Fix:

```
async function reloadAndRefreshAll() {
  await loadUserTasks(currentUser.username);
  await loadScheduleEntries();
  updateTasksDisplay();
  renderCurrentScheduleView();
  renderTimeline(scheduleEntries);
  refreshDashboardIfVisible();
}
```

With this change, all three duplicate blocks can be replaced with a single call to reloadAndRefreshAll().

Smell 3:
Magic Constants: hardcoded default values scattered throughout
Category: Dispensable / Primitive Obsession

Description: Numeric and string defaults like "09:00", "21:00", 3, 1, "MEDIUM", and "#6366f1" are hardcoded in multiple places such as exitEditMode(), editSelectedTask(), addTask(), and inside loadUserTasks(). As of now, if a default changes, every occurrence must be updated manually.

Current code:

```
if (preferredStartTime) preferredStartTime.value = "09:00";  // in
exitEditMode
preferredStartTime: task.preferredStartTime || "09:00",       // in
loadUserTasks
if (taskMaxHoursPerDay) taskMaxHoursPerDay.value = "3";       // in
exitEditMode
maxHoursPerDay: task.maxHoursPerDay ?? 3,                     // in
loadUserTasks
```

Proposed Fix: Extract and initialize as constants:

```
const DEFAULTS = {
  PREFERRED_START_TIME: "09:00",
  PREFERRED_END_TIME: "21:00",
  MAX_HOURS_PER_DAY: 3,
  MIN_BLOCK_HOURS: 1,
  MAX_BLOCK_HOURS: 3,
  DIFFICULTY: "MEDIUM",
  TASK_COLOR: "#6366f1",
};
```

With this code, all hardcoded usages can be replaced with references to DEFAULTS.PREFERRED_START_TIME, etc. This helps organize and eliminates any risk of inconsistency within the code.

Smell 4:
Inconsistent Level of Abstraction — renderTimeline inline HTML vs helpers

Category: Bloater / Preventers

Description: renderTimeline and renderScheduleGrid build long HTML strings with template literals inline which include conditional logic, style expressions, and color mapping objects such as `({ LOW: "#10b981", MEDIUM: "#f59e0b", HIGH: "#ef4444" })`, that are directly embedded in template strings. This mixes data formatting and styling logic in one place, making it hard to maintain.

Proposed Fix: Extract helper functions:

```
function getDifficultyColor(difficulty) {
  const map = { LOW: "#10b981", MEDIUM: "#f59e0b", HIGH: "#ef4444" };
  return map[difficulty] || "#64748b";
}

function renderEventCard(event) {
  return `<div class="schedule-event ${event.type}" ...>...</div>`;
}
```

Extracting these helpers helps in making each rendering concern readable and reusable across both weekly and monthly grids.

Committed changes to GitHub:
https://github.com/hvpham-yorku/Group-5-ScheduleLynx/tree/Beerkamal-Bhagal-Code-Smells-Fix